

Data augmentation

Make extra training examples from the ones you have, using changes that keep the label, then check that they help.

THE QUESTION

The labelling budget is spent and the model still overfits. How do you get more training examples without labelling any?

THE INTUITION

Apply changes the label should not care about: a mirror image, a small tilt, dimmer light, background noise. The model sees many views of each example and learns to ignore those changes.

KEY TAKEAWAY

A transform is safe only if it keeps the label. Augment the training split alone, and keep a transform only if clean validation accuracy goes up.

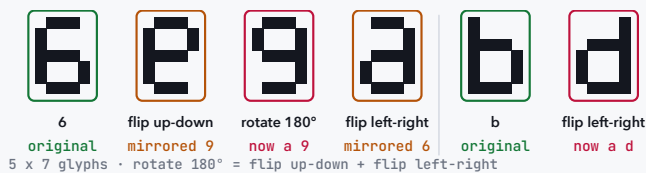
labelled example → random transforms, each with p → new view, same label → train → clean validation → keep or drop

1 Each transform teaches one invariance



A network does not know that a flipped cat is still a cat until it sees one. Geometric transforms mimic viewpoints, colour ones lighting, noise and blur the sensor, cutout a hidden part. An image never seen twice the same way is hard to memorise, so the train/test gap shrinks.

2 Ask whether the change keeps the label



For cats and dogs, left-right flips, colour jitter and greyscale are safe; an up-down flip or a 180° turn is not. The domain decides:

medical scans mild rotation, brightness; no flips (a flipped chest X-ray puts the heart on the wrong side)

satellite documents any rotation, colour shifts

digits slight perspective; no turns over 5° turns under 15°; no up-down flip

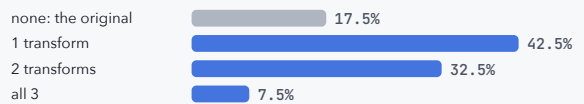
3 Mixup and CutMix mix the labels too



$$\tilde{x} = \lambda x_A + (1 - \lambda) x_B, \quad \tilde{y} = \lambda y_A + (1 - \lambda) y_B$$

Mixup draws λ from $\text{Beta}(\alpha, \alpha)$ (the lab uses $\alpha = 0.4$). CutMix pastes a patch of B into A and weights labels by area. The lecture's advice: mixup for general classification, CutMix when spatial layout matters. Cutout blanks a patch and keeps the label, so the model must use every part of the object.

4 A pipeline is a chain of coin flips

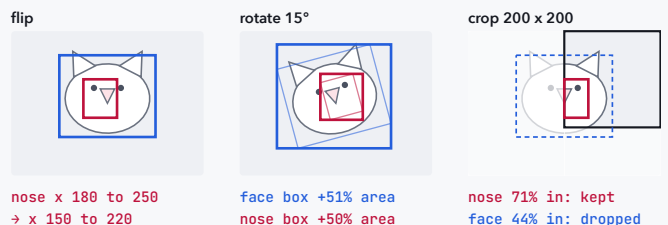


$p = 0.5, 0.5, 0.3$ as in the code · 1.3 transforms per draw on average

Each transform fires with its own p , redrawn every time the loader returns the image. Here 17.5% of draws come out unchanged.

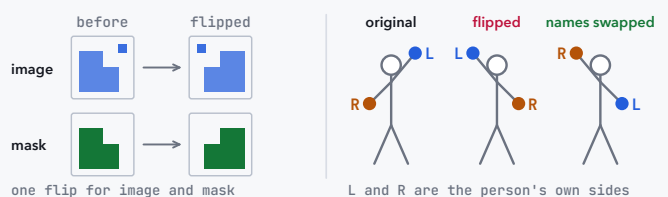
```
import augmentations as A
train_tf = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.5),
    A.RandomBrightnessContrast(p=0.3)])
x = train_tf(image=img)["image"]
```

5 Boxes must move with the image



A flip of width W sends $x_{\min} \rightarrow W - x_{\max}$. A turn re-fits the box around the turned corners, here with half as much area again; a crop can cut a box, and min_visibility drops it. Albumentations does all three given bbox_params and the format: `pascal_voc` $[x_0, y_0, x_1, y_1]$ pixels, `coco` $[x_0, y_0, w, h]$, `yolo` centre and size in 0 to 1 (the face box is $[0.5, 0.45, 0.5, 0.567]$).

6 Masks and keypoints move too



Pass the mask with the image, `tf(image=img, mask=m)`, so every pixel keeps its class. After a flip the left wrist becomes a right one: swap the names.

Keep the meaning, then measure

Text and audio need their own transforms. Whatever the data, compare against a baseline on clean validation data.

7 One word can flip a sentence's label

original	the	movie	was	not	bad	at	all	POS	
synonym	the	film	was	not	bad	at	all	POS ok	
swap	the	movie	not	was	bad	at	all	POS ok	
insert	the	movie	was	really	not	bad	at	all	POS ok
delete	the	movie	was	—	bad	at	all	NEG !	
antonym	the	movie	was	not	good	at	all	NEG !	

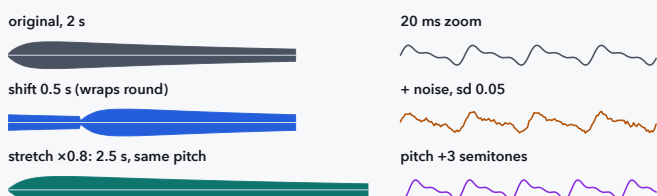
illustrative review, label POS · the last two rows flip the label

illustrative review, Label POS · the last two rows flip the label

Easy data augmentation (EDA) has four operations: synonym replacement, random insertion, swap and deletion. Back-translation (English to German and back) rephrases more naturally; an LLM paraphrases best but costs API calls. For named-entity tagging, list the entities as stopwords so they never change.

```
aug = naw.SynonymAug(aug_src="wordnet",
stopwords=["John", "Google"])
```

8 Sound: shift, stretch, noise, pitch

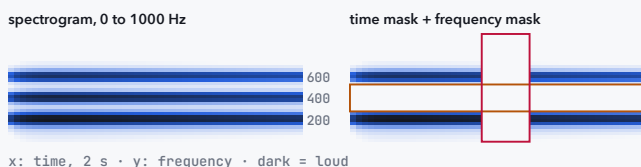


the demo's tone: 200 + 400 + 600 Hz, 16 kHz · pitch: 200 → 238 Hz

speech noise, reverb, speed; mild pitch only
speaker ID noise, reverb; pitch alters the voice
music genre stretch, noise; pitch alters the key
emotion noise, reverb; speed alters the mood

```
aug = Compose([AddGaussianNoise(p=0.5),
TimeStretch(0.8, 1.25, p=0.5),
PitchShift(-4, 4, p=0.5)])
y = aug(samples=audio, sample_rate=16000)
```

9 SpecAugment masks time and frequency



x: time, 2 s · y: frequency · dark = loud

A time mask hides a run of frames, so the model uses context; a frequency mask hides a band, so it cannot lean on one pitch range. Here 8 bins of 31.25 Hz and 40 frames of 8 ms are zeroed; the parameters below set the largest width drawn.

```
spec = TimeMasking(time_mask_param=40)(
FrequencyMasking(freq_mask_param=30)(spec))
```

10 Measured: this run lost accuracy



From the saved output of the CIFAR-10 demo notebook (MLX version): one CNN (667,818 parameters), 5,000 training images, 30 epochs. Augmentation cut the train/test gap at epoch 30 from 8.4 to 3.0 points but lowered the best test accuracy from 56.1% to 46.7%. The baseline reached only 64% on its own training images, so it was not memorising, and both runs were still climbing. Augmentation is a regulariser: it helps a model that overfits and can hurt one that underfits or trains too briefly.

11 Augment the training split only

train	validation	test
new random view every epoch	clean, the same every time	clean, the same every time

test-time augmentation: predict on the original and 4 flipped or shifted views

0.58 0.65 0.71 0.67 0.64 → mean 0.65

P(cat) per view, illustrative · 5 views cost 5 forward passes

Validation and test data must look like real data.

Test-time augmentation (TTA) is a deliberate exception: average the predictions over several views of one input. The lecture quotes 1 to 2 points gained for N times the inference cost.

12 Tune strength like any hyperparameter

Start with $p = 0.5$, a small magnitude and 2 to 3 transforms. Add one at a time, retrain, keep what helps. Raise the magnitude if the model still overfits; lower it if validation accuracy drops.

RandAugment(num_ops=2, magnitude=9) picks transforms at random.

under 1,000	heavy, about 10 versions each
1,000 to 10,000	moderate, about 5
10,000 to 100,000	light, 2 to 3
over 100,000	little or none

Too strong? Label 100 augmented samples yourself. If fewer than 95 are clearly the right class, weaken it.

Libraries: Albumentations and torchvision (images), nlpaug (text), audiomentations and torchaudio (audio), AugLy (all modalities).

keep the label · move boxes, masks and keypoints too · training data only · measure on clean data

Source: lecture "Data augmentation", its demos (CIFAR-10 impact notebooks, object detection with Albumentations, text and audio augmentation) and its lab, nipunbatra.github.io/stt-ai-teaching. Toy images, glyphs, the review and the TTA numbers are illustrative; the boxes (a 400 x 300 frame), the tone and the CIFAR-10 curve come from the demos. Cheap ways to get the labels in the first place are on the "Labelling less" sheet.